Hyperparameter Tuning

K-Nearest Neighbors & Decision Trees

A complete tutorial — from the very first idea to pro-level grid search

BASIC	INTERMEDIATE	PRO
ideas, analogies, hand-made examples	the math, every hyperparameter	pruning theory, efficiency, pipelines

Dark-theme edition · every chart in this book comes from code that was actually run

Contents

Contents 2

How to read this tutorial 4

PART 0 — Foundations 5

0.1 Parameters vs hyperparameters . . . 5
0.2 Underfitting and overfitting . . . 5
0.3 Train / test split and cross-validation . . . 6
0.4 Grid search . . . 6

PART 1 — K-Nearest Neighbors (KNN) 7

1.1 The big idea: “tell me who your neighbours are” . . . 7
1.2 n_neighbors — the “K” in KNN . . . 9
1.3 weights — how much voting power each neighbour gets . . . 11
1.4 metric and p — how “distance” is measured . . . 13
1.5 algorithm — the search engine (speed only) . . . 14
1.6 Scaling — the rule you must never forget . . . 15
1.7 Putting it together: the Grid Search setup . . . 17
1.8 Pro level: KNN from scratch (≈15 lines) . . . 18
1.9 KNN cheat-sheet . . . 19

PART 2 — Decision Trees 20

2.1 The big idea: a flowchart that writes itself . . . 20
2.2 How the tree chooses its questions (the math) . . . 21
2.3 The core problem: trees memorise . . . 22
2.4 Pre-Pruning (hyperparameter tuning) . . . 22
2.5 Post-Pruning: Cost-Complexity Pruning (ccp_alpha) . . . 24
2.6 Data preparation strategy . . . 25
2.7 Handling class imbalance . . . 26
2.8 Efficiency tricks from the community (Stack Overflow & Reddit) . . . 27
2.9 The complete tuned pipeline — your code, explained line by line . . . 30
2.10 Pro toolbox: look inside your tree . . . 33
2.11 Decision Tree cheat-sheet . . . 33

PART 3 — Wrap-up 34

3.1 KNN vs Decision Tree at a glance 34
3.2 Ten mistakes to avoid 34
3.3 Practice exercises 34
Appendix — coverage checklist of your original script 35

How to read this tutorial

Your original material was a video script: correct content, but unordered and without pictures. I reorganised it into a learning path and added the missing pieces — visuals, the underlying math, worked examples, and real experiments. **Nothing from your script was removed**; the original sentences appear inside the orange "From your script" boxes and in the sections where they belong (a coverage checklist is in the appendix).

The learning path

Part	What you learn	Level
0 Foundations	Hyperparameters, overfitting vs underfitting, cross-validation, grid search	Basic
1 KNN	The algorithm by hand, the math of distances, n_neighbors, weights, metric/p, algorithm, scaling, full grid search, from-scratch code	Basic → Pro
2 Decision Trees	Anatomy, Gini/entropy, information gain, pre-pruning, cost-complexity pruning, data prep, imbalance, community efficiency tricks, full tuned pipeline	Basic → Pro
3 Wrap-up	KNN vs Decision Tree comparison, common mistakes, practice exercises, coverage checklist	All

Legend

✎ From your script
Original text, kept in full.

☺ Everyday analogy
A real-life picture to make the idea stick.

Math appears in yellow boxes like this

★ Tip
Practical advice you can use immediately.

▲ Watch out
A common mistake that silently ruins results.

◆ Real run (executed while building this tutorial)
Numbers produced by actually running the code while building this tutorial.

PART 0 — Foundations

Four ideas you need before tuning anything

0.1 Parameters vs hyperparameters

● LEVEL: BASIC

A model has two kinds of numbers:

	Parameters	Hyperparameters
Who sets them?	The algorithm **learns** them from data.	**You** choose them before training.
Examples	Split thresholds inside a tree (Salary > 50,000); the stored training points of KNN.	K in KNN, max_depth of a tree, ccp_alpha.
Analogy	The recipe the chef learned by tasting.	The oven temperature you pick before cooking.

Tuning means trying many oven temperatures and keeping the one that makes the best cake — measured on cakes the chef has *not* tasted yet.

0.2 Underfitting and overfitting

● LEVEL: BASIC

Underfitting = the model is too simple to see the pattern (high *bias*). **Overfitting** = the model memorises noise and quirks of the training set (high *variance*) and fails on new data. Almost every hyperparameter in this tutorial is a dial between these two extremes.

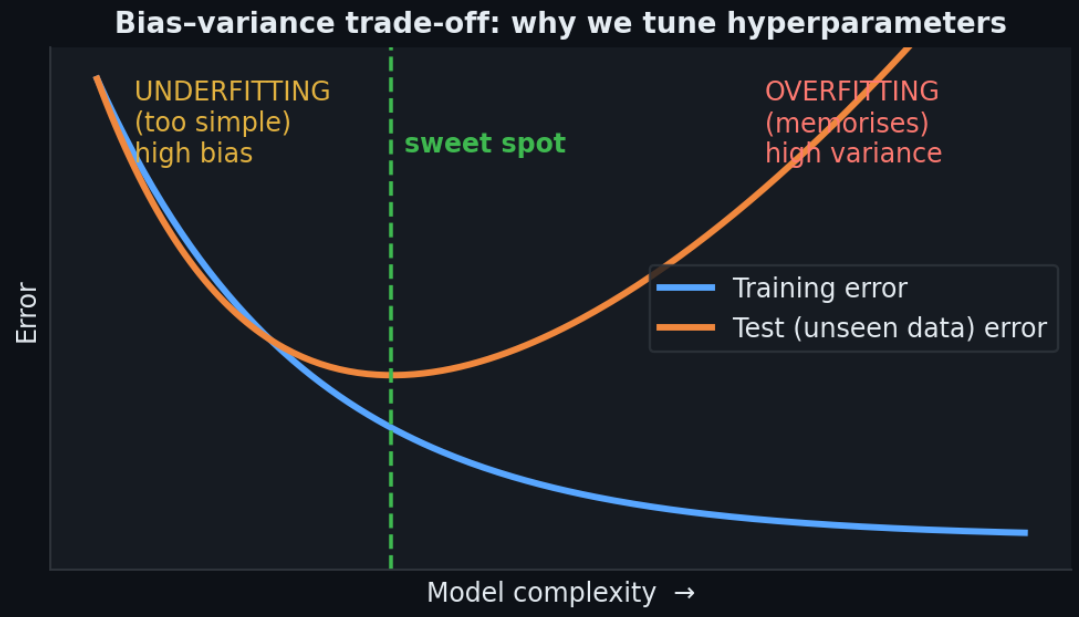


Figure 0.1 — Training error keeps falling as the model gets more complex, but error on unseen data has a U-shape. We want the bottom of the U.

$$\text{Expected test error} \approx \text{Bias}^2 + \text{Variance} + \text{Irreducible noise}$$

Bias = error from wrong/too-simple assumptions. Variance = error from being too sensitive to the particular training sample. Noise = randomness no model can remove.

0.3 Train / test split and cross-validation

● LEVEL: BASIC

We never grade a model on the same data it studied. First, lock away a **test set** (e.g. 20%) and never touch it while tuning. Then, to compare hyperparameter choices fairly, we use **k-fold cross-validation** on the training part: split it into 5 folds, train on 4, validate on the 1 left out, rotate 5 times, and average the 5 scores.

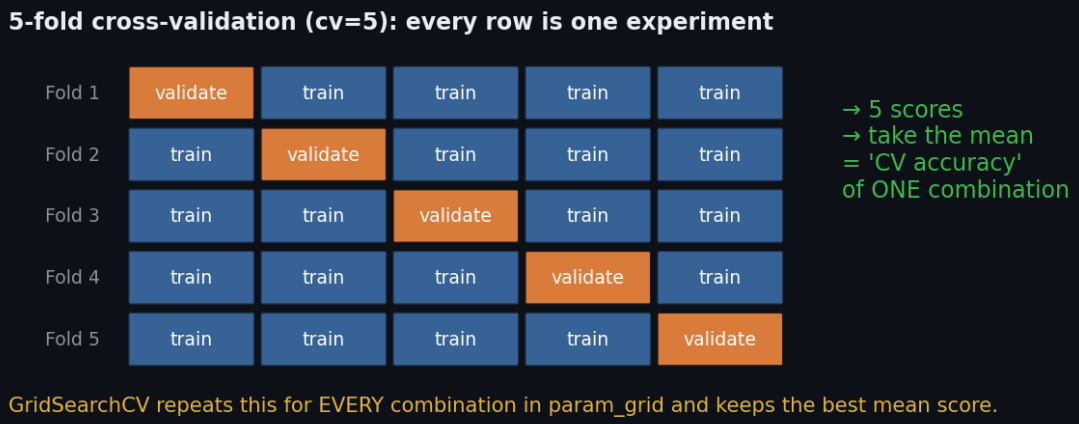


Figure 0.2 — 5-fold cross-validation. Every sample is used for validation exactly once.

0.4 Grid search

● LEVEL: BASIC

Grid search simply tries **every combination** from lists you provide. The total work is easy to compute:

number of fits = (values of param 1) × (values of param 2) × … × cv folds

KNN grid: 7 × 2 × 2 = 28 combinations × 5 folds = 140 fits. Decision-tree grid: 5 × 4 × 4 × 3 × 3 = 720 × 5 = 3,600 fits.

the grid-search pattern used throughout

```
from sklearn.model_selection import GridSearchCV
gs = GridSearchCV(estimator=model, param_grid=param_grid, cv=5, scoring='accuracy', n_jobs=-1)
gs.fit(X_train, y_train)
gs.best_params_      # the winning combination
gs.best_score_       # its mean cross-validation score
gs.best_estimator_   # the model refit on all training data with those values
```

PART 1 — K-Nearest Neighbors (KNN)

Hyperparameter tuning from the very first idea to a production-quality grid search

1.1 The big idea: “tell me who your neighbours are”

● **LEVEL: BASIC**

> ☺ **Everyday analogy**
>
> You move to a new city and wonder if a restaurant is good. You ask the **3 people living closest** to it. Two say “great”, one says “bad”. You trust the majority: **good**. That is KNN. There is no formula to learn and no training in the usual sense — the algorithm just **remembers all the data** and, when asked, looks at the closest examples. (This is why KNN is called a *lazy learner*.)

The algorithm in 5 steps

Step	What happens	Controlled by hyperparameter
1	Store all training points (with their labels).	—
2	A new point arrives. Measure its distance to **every** stored point.	**metric** / **p**
3	Pick the **K** closest points (the neighbours). Search method = how fast we find them.	**n_neighbors**, **algorithm**
4	Let the neighbours vote (classification) or average (regression). Each vote may count equally or by closeness.	**weights**
5	Return the winner / the average as the prediction.	—

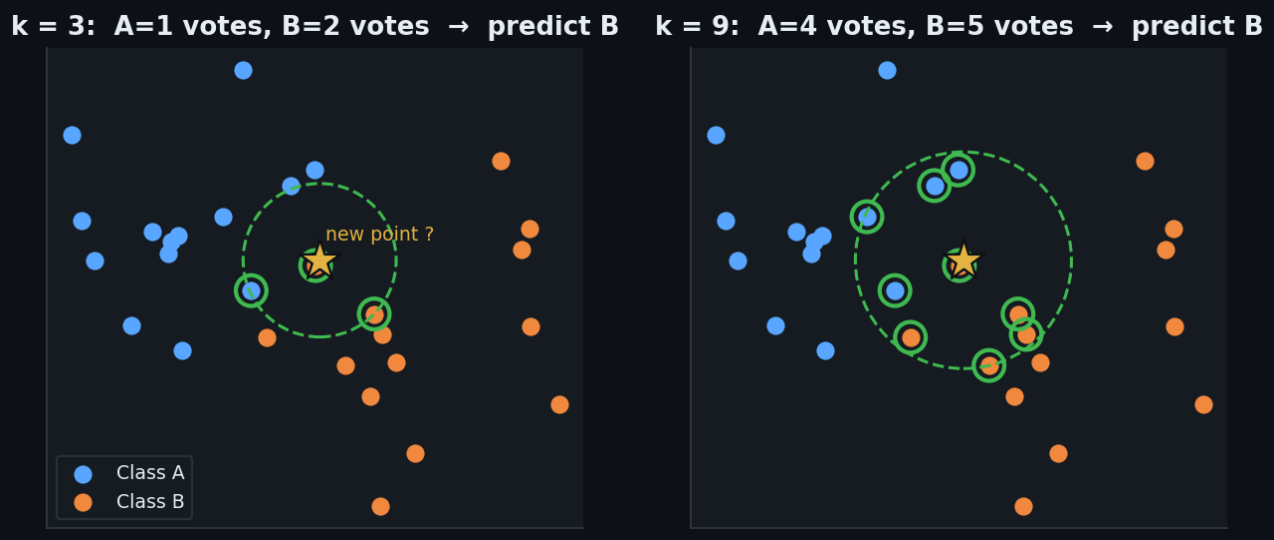


Figure 1 — The yellow star is the new point. With k=3 the green circle contains 2 neighbours of one class; with k=9 the circle is bigger and the vote can flip.

Math you need (it is small!)

Distance between two points. Take two points with *d* features: $a = (a_1,\ldots,a_d)$ and $b = (b_1,\ldots,b_d)$.

$$\text{Euclidean } (p = 2): d(a,b) = \sqrt{\sum_i (a_i - b_i)^2}$$

$$\text{Manhattan } (p = 1): d(a,b) = \sum_i |a_i - b_i|$$

$$\text{Minkowski (any } p): d(a,b) = \left(\sum_i |a_i - b_i|^p\right)^{1/p}$$

Minkowski is the "parent" formula. Put p=1 and you get Manhattan; put p=2 and you get Euclidean — that is exactly why sklearn only needs the single number p.

Classification: $\hat{y}$ = the class with the most votes among the K neighbours

Regression: $\hat{y} = (1/K) \cdot \sum_{j=1..K} y_j$

Distance-weighted: $w_j = 1 / d_j \rightarrow \hat{y} = \sum w_j \cdot y_j / \sum w_j$ (class score = sum of weights of that class)

Worked example (do it by hand once!)

Training data (2 features) and a new point **Q = (3, 3)**:

Point	(x, y)	Class	Euclidean distance to Q	Manhattan distance to Q	Rank
P1	(1, 1)	Red	√(4+4) = 2.83	2 + 2 = 4	4
P2	(2, 3)	Red	√(1+0) = 1.00	1 + 0 = 1	1
P3	(4, 4)	Blue	√(1+1) = 1.41	1 + 1 = 2	2
P4	(5, 2)	Blue	√(4+1) = 2.24	2 + 1 = 3	3
P5	(6, 5)	Blue	√(9+4) = 3.61	3 + 2 = 5	5

- **k = 1** → nearest is P2 (Red) → predict **Red** (one single neighbour decides everything — risky).
- **k = 3** → P2 (Red), P3 (Blue), P4 (Blue) → 1 Red vs 2 Blue → predict **Blue**.
- **k = 5** → 2 Red vs 3 Blue → predict **Blue**.
- **k = 3 with weights='distance'** → Red score = 1/1.00 = 1.00; Blue score = 1/1.41 + 1/2.24 = 0.71 + 0.45 = 1.15 → still **Blue**, but the margin is now 1.15 vs 1.00 instead of 2 vs 1. The very close Red point pulls much harder.

> **● Note**
> Notice how the answer changed between k=1 and k=3. Choosing K is the most important decision in KNN — the next section is all about it.

1.2 n_neighbors — the “K” in KNN

● LEVEL: BASIC

✎ From your script — kept in full

What it is: The number of closest data points the algorithm looks at to make a prediction. If k=5, the model checks the 5 nearest points and takes a majority vote (for classification) or an average (for regression). Small values make the model highly sensitive to noise, while very large values blur the decision boundaries.

How it works: This is the core “K” in K-Nearest Neighbors. It defines exactly how many adjacent data points the algorithm will look at to make a prediction. If you set it to 5, the model finds the 5 physically closest data points in the training set and takes a majority vote for classification, or calculates a mean average for regression.

How it helps: It acts as the primary dial for controlling model complexity. By adjusting this parameter, you force the algorithm to either look at a hyper-local area (just the immediate neighbors) or a broader, more generalized area to make its decisions.

How it affects the model

	Small K (e.g. 1 or 3)	Large K (e.g. 20 or 50)
Behaviour	Highly sensitive to local data and background noise. Highly complex, jagged decision boundaries.	Looks at a massive neighbourhood, smoothing out noise but also blurring over genuine, underlying patterns.
Result	**Overfitting** (low bias, high variance). Memorises the training data but struggles with new, unseen data.	**Underfitting** (high bias, low variance). Over-generalises and draws overly simple decision boundaries.
Analogy	Asking only your 1 closest neighbour — one weird person ruins the answer.	Asking the whole city — you lose the local flavour.

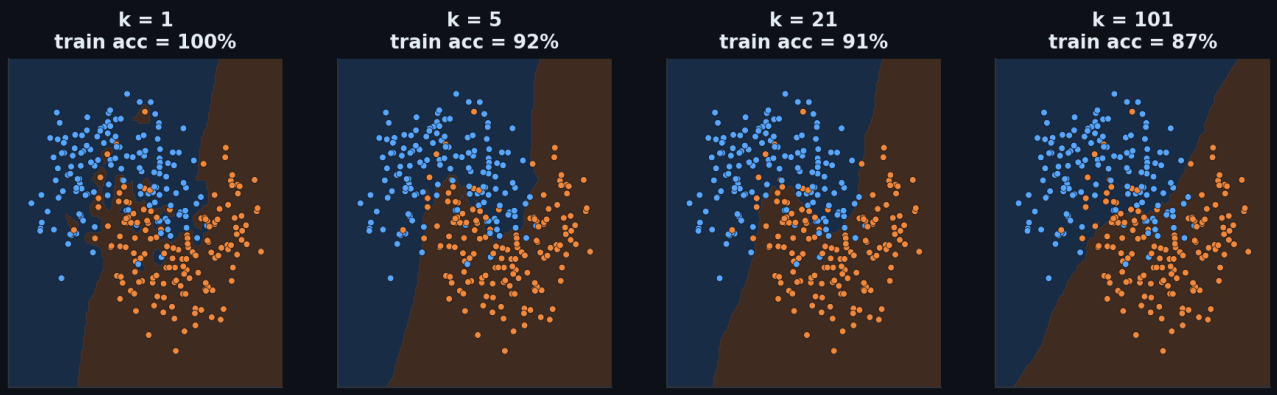


Figure 2 — Decision regions on the same data. k=1: jagged islands around single noisy points (100% train accuracy = memorisation). k=101: almost one smooth line, details lost.

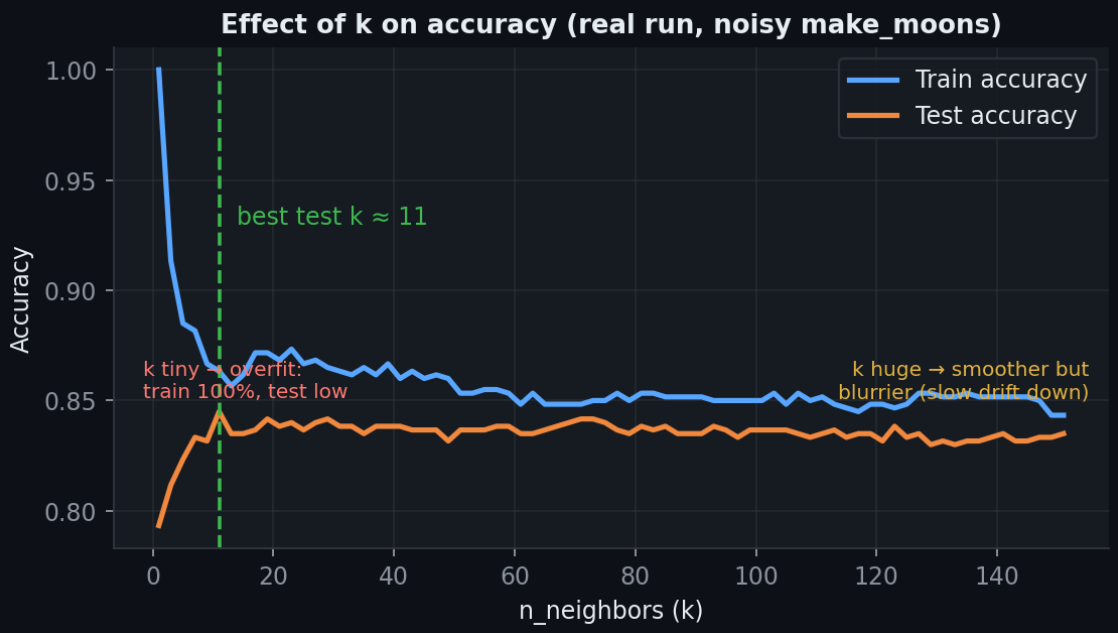


Figure 3 — A real experiment. At k=1 training accuracy is a perfect 100% but test accuracy is the lowest; it recovers around k≈11. For large k the curves flatten and drift down (the model is too blurry). The best K is found with cross-validation, not by eye.

Rule of thumb for a first guess: K ≈ √N (N = number of training samples)

Only a starting point — always confirm with grid search.

Values for Grid Search

Preferred (standard): `[3, 5, 7, 9, 11, 15, 21]`. **Extended (when you have a lot of data):** `[3, 5, 7, 9, 11, 15, 21, 31]`.

★ Tip

Always prioritise odd numbers for binary classification to prevent a 50/50 tie vote between the two classes (with k=4 you can get 2 vs 2). **Avoid k=1** in your grid: it simply echoes the immediate neighbour and almost always overfits by memorising the training data.

● Pro note

Pro detail: with 3 or more classes an odd K does *not* guarantee the absence of ties (e.g. k=5 can split 2-2-1... or k=3 can split 1-1-1). Scikit-learn then resolves ties deterministically (not randomly), which is arbitrary — another reason to use weights='distance', which makes exact ties extremely unlikely.

1.3 weights — how much voting power each neighbour gets

● LEVEL: BASIC

✎ From your script — kept in full

What it is: Determines how much “voting power” each of those neighbors gets.

uniform: Every neighbor gets an equal vote. A point 10 meters away counts exactly the same as a point 1 meter away. (Every neighbor within the “K” count gets an equal, identical vote. A point located right next to the target has the exact same influence as a point on the furthest edge of the neighborhood boundary.)

distance: Closer neighbors have a stronger influence on the final prediction than neighbors that are further away. (Closer neighbors are given a heavier, disproportionate weight — inversely proportional to their physical distance. A very close point has a massive say in the final prediction, while a distant point only contributes a fraction of a vote.)

$$\text{uniform: } w_j = 1 \quad \text{distance: } w_j = 1 / d_j$$

$$\text{score(class c)} = \Sigma_{j \text{ in neighbours of class c}}\ w_j \rightarrow \text{predict the class with the largest score}$$

A concrete example

Five neighbours: two of class A at distances 0.94 and 1.17, and three of class B at distances 3.16, 3.30 and 3.44.

Method	Class A score	Class B score	Prediction
uniform	2 votes = 40%	3 votes = 60%	**B** (majority wins, even though B is far away)
distance	1/0.94 + 1/1.17 = 1.06 + 0.86 = 1.92 → 68%	1/3.16 + 1/3.30 + 1/3.44 = 0.32+0.30+0.29 = 0.91 → 32%	**A** (the two very close points override the three far ones)

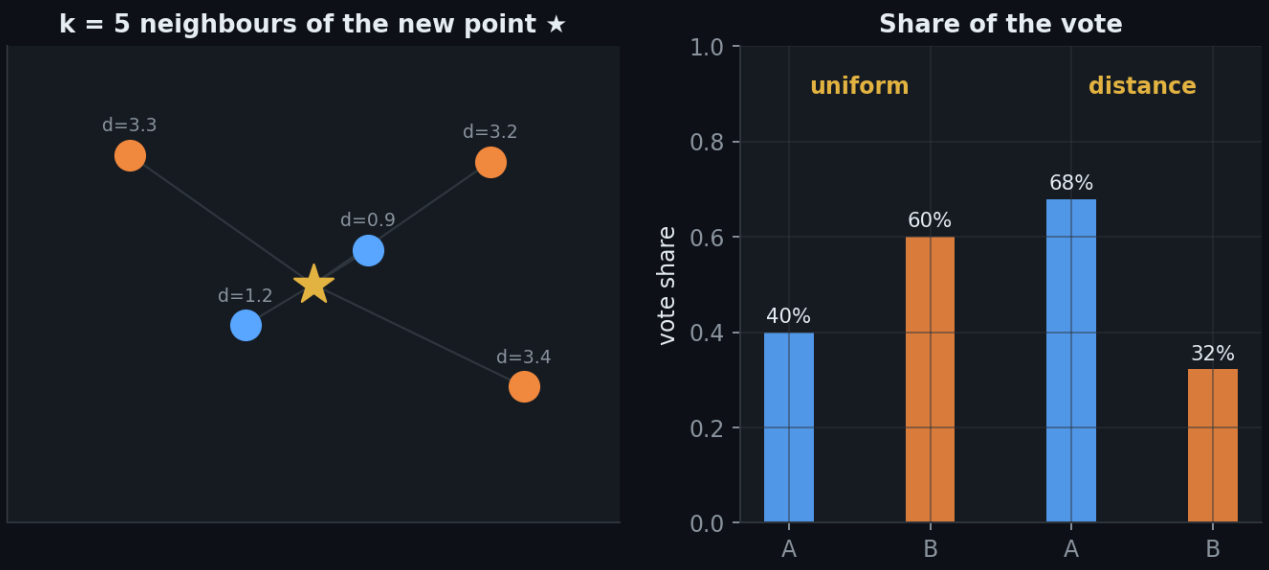


Figure 4 — Left: the new point ★ and its 5 neighbours with distances. Right: the share of the vote under each weighting. Same data, different answer.

How it helps

It solves tie-breakers and prevents densely packed, overlapping classes from overwhelming a prediction. If your data is highly irregular or clusters are messy, weighting by distance ensures that **immediate proximity overrides sheer volume**.

How it affects the model

Using *distance* increases the model's complexity and local sensitivity slightly. It is specifically beneficial for **imbalanced datasets** where one class might naturally outnumber another in a given region; the distance weight pulls the prediction toward the absolute closest points regardless of what the rest of the neighborhood looks like.

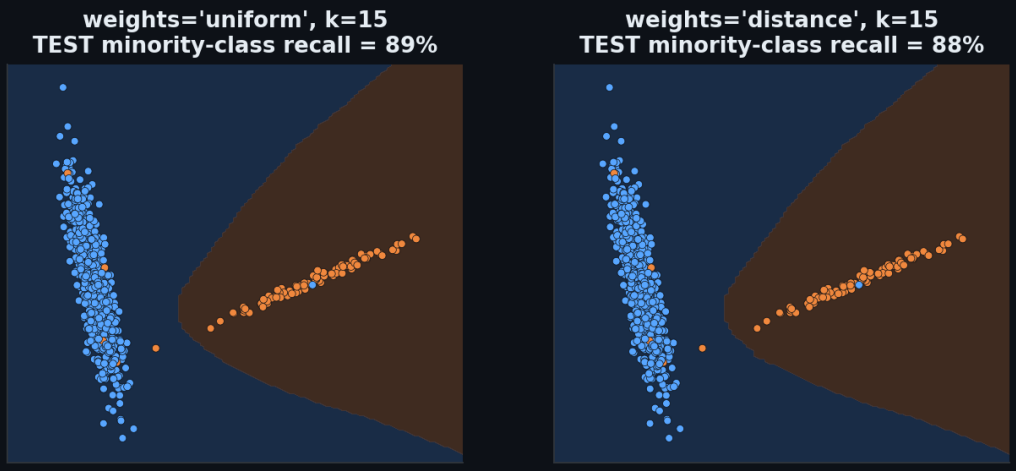


Figure 5 — A 90/10 imbalanced dataset, k=15, scored on a held-out test set. Honest result: here the two settings are almost tied (minority recall about 89% vs 88%). Distance weighting is a strong candidate for imbalanced or messy data, not a guaranteed win — which is exactly why it sits in the grid and cross-validation decides.

> **▲ Pro warning**
>
> With weights='distance' a training point that is *exactly* on top of the query gets distance 0. Scikit-learn handles this safely (the exact match wins), but it also means that when you evaluate on the **training set**, distance-weighted KNN scores ≈100% by construction. Always judge on validation/test data.

Values for Grid Search: `['uniform', 'distance']`

1.4 metric and p — how “distance” is measured

● **LEVEL: INTERMEDIATE**

✎ From your script — kept in full

What it is: The mathematical formula used to calculate the physical space between data points. In Python's Scikit-Learn, the default metric is *minkowski*, which is controlled by the hyperparameter p. (Because KNN makes predictions based entirely on the spatial gap between points, it needs a mathematical formula to measure that distance. The Minkowski distance serves as a generalised formula controlled by the power parameter p.)

p = 1 (Manhattan Distance): Calculates distance on a grid, like a taxi driving through city blocks (strictly along a grid axis). Often performs better when you have a high number of features (high-dimensional data).

p = 2 (Euclidean Distance): Calculates the straight-line distance, “as the crow flies” (the direct distance between two points). Best for standard, continuous numerical data.

How it helps: It allows the algorithm's geometry to adapt to the specific dimensionality and shape of your data features.

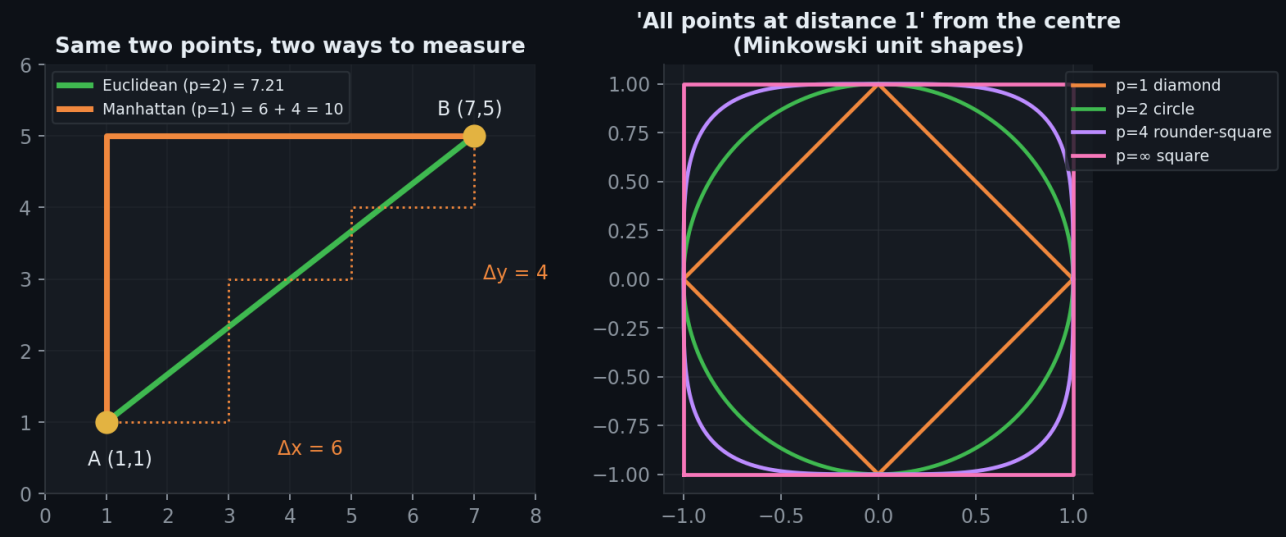


Figure 6 — Left: A→B is 7.21 units “as the crow flies” (Euclidean) but 10 units for a taxi (Manhattan). Right: the set of points exactly 1 away from the centre is a diamond for p=1, a circle for p=2 and approaches a square as p grows — different p means different idea of “nearby”.

How it affects the model

Euclidean distance (p=2) works exceptionally well for standard, continuous numeric variables in lower dimensions. However, as you add more features (columns), all points begin to mathematically look equidistant from one another — a phenomenon known as the **curse of dimensionality**. Manhattan distance (p=1) is significantly more robust to outliers and maintains its predictive integrity much better when your dataset has high dimensionality.

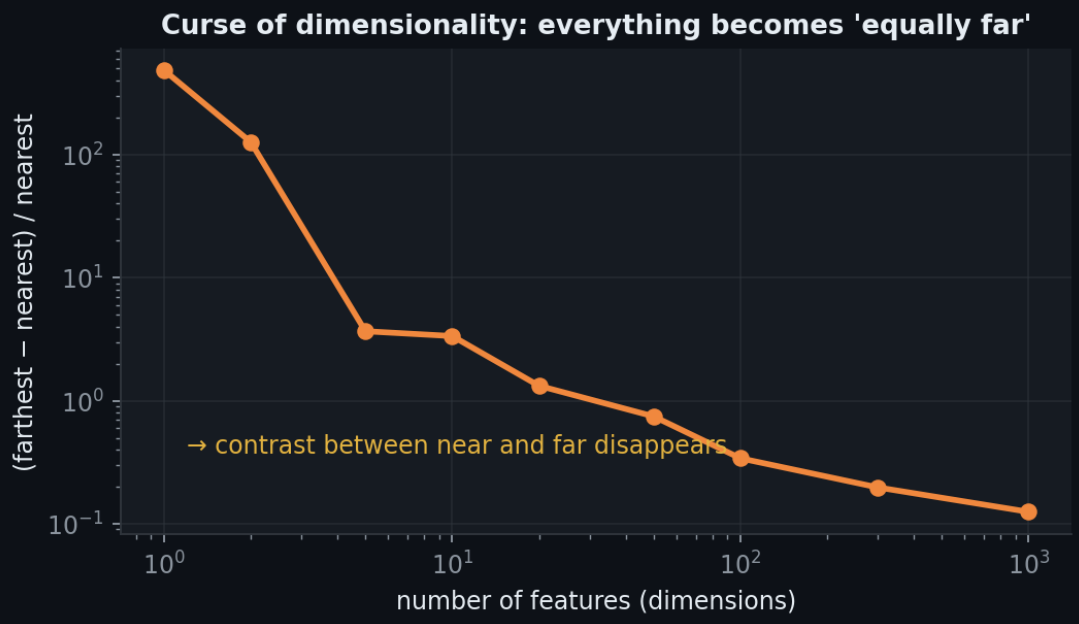


Figure 7 — Real simulation: 500 random points, one query. The relative gap between the farthest and the nearest point shrinks by orders of magnitude as dimensions grow. When "near" and "far" look alike, a nearest-neighbour vote becomes almost random.

● Why it works

Why is Manhattan more outlier-robust? Euclidean distance *squares* every coordinate difference, so one feature with a huge gap (an outlier) dominates the total. Manhattan adds the gaps linearly, so a single extreme feature has less leverage. (In practice the gain is dataset-dependent — which is exactly why p goes into the grid and we let cross-validation decide.)

Values for Grid Search: `p ∈ [1, 2]`

● Pro note

In scikit-learn, p is only used when `metric='minkowski'` (the default). Other metrics exist (e.g. 'cosine', 'chebyshev', 'hamming' for categorical bits) — advanced users can add them to the grid, but p∈[1,2] covers 90% of tabular problems.

1.5 algorithm — the search engine (speed only)

● LEVEL: INTERMEDIATE

✎ From your script — kept in full

What it is: The underlying indexing method the computer uses to search for the nearest neighbors (ball_tree, kd_tree, brute, auto).

Preferred Grid Search Values: Exclude from Grid Search (leave as 'auto').

Tip: This hyperparameter affects computation speed, not the accuracy of your model's predictions. Leaving it on 'auto' allows the algorithm to automatically select the most efficient search method based on the shape of your training data.

Value	How it finds neighbours	Best when
brute	Computes the distance to **every** training point (the plain method from section 1.1). Cost per query ≈ N × d.	Small data, or very many features (> ~15–20) where trees stop helping

Value	How it finds neighbours	Best when
kd_tree	Splits space with axis-aligned cuts into a tree; skips whole regions that cannot contain a neighbour.	Low dimensions (few features), lots of rows
ball_tree	Groups points into nested spheres (“balls”); skips balls that are too far away.	Medium dimensions, awkward distance metrics
auto	Scikit-learn picks one of the above from the data shape (roughly: brute if there are many features or K is large vs N, otherwise a tree).	**Default — leave it**

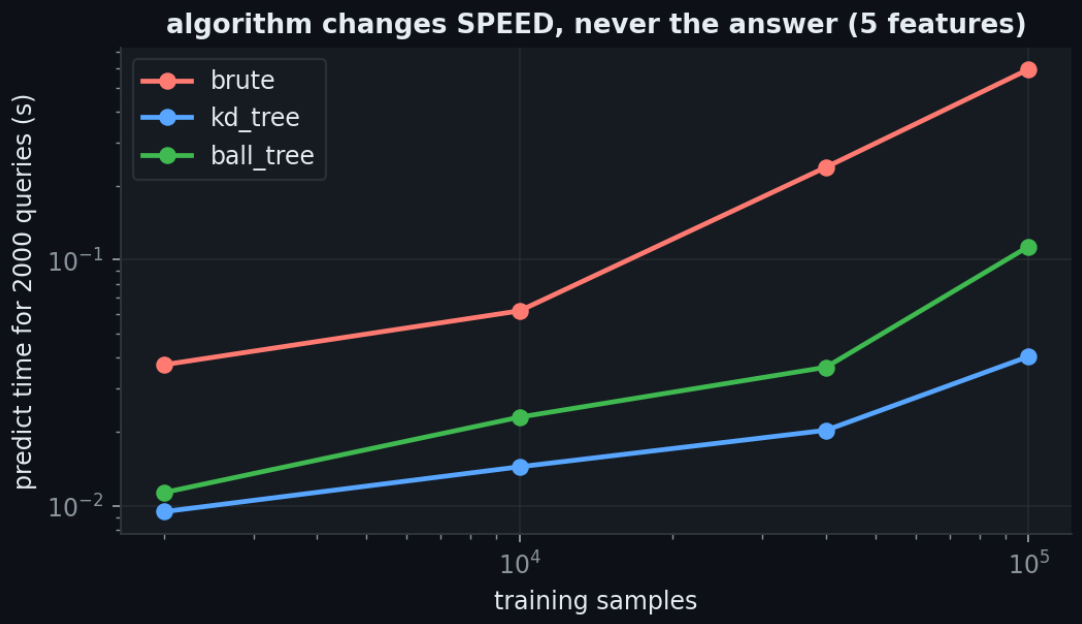


*Figure 8 — Real timing (5 features). The tree searches are many times faster than brute force as the data grows. I also verified on 5,000 points that all three methods return **exactly the same predictions** — the algorithm changes speed, never accuracy.*

> **★ Tip**
> That is why it is excluded from the grid: adding it would multiply your search time by 3–4× and give you zero accuracy information.

1.6 Scaling — the rule you must never forget

● LEVEL: BASIC

Because this algorithm relies exclusively on calculating distances, you **must scale your data** (using tools like `StandardScaler` or `MinMaxScaler`) before executing the grid search. If one feature is measured in thousands (like salary) and another in single digits (like age), the larger numbers will completely dominate the distance math, rendering your hyperparameter tuning useless.

Why? A numeric example

Query person: age 32, salary 60,000. Candidate 1: age 30, salary 50,000. Candidate 2: age 55, salary 52,000.

$$d(Q, C1) = \sqrt{(32-30)^2 + (60000-50000)^2} = \sqrt{4 + 100{,}000{,}000} \approx 10{,}000.0002$$

$$d(Q, C2) = \sqrt{(32-55)^2 + (60000-52000)^2} = \sqrt{529 + 64{,}000{,}000} \approx 8{,}000.03$$

Raw units say C2 is “closer” — even though that person is 23 years older! The age difference (2 vs 23) is invisible next to the salary difference.

StandardScaler: $z = (x - \mu) / \sigma$ (mean 0, standard deviation 1)

MinMaxScaler: $x' = (x - x_{min}) / (x_{max} - x_{min})$ (every feature squeezed into 0…1)

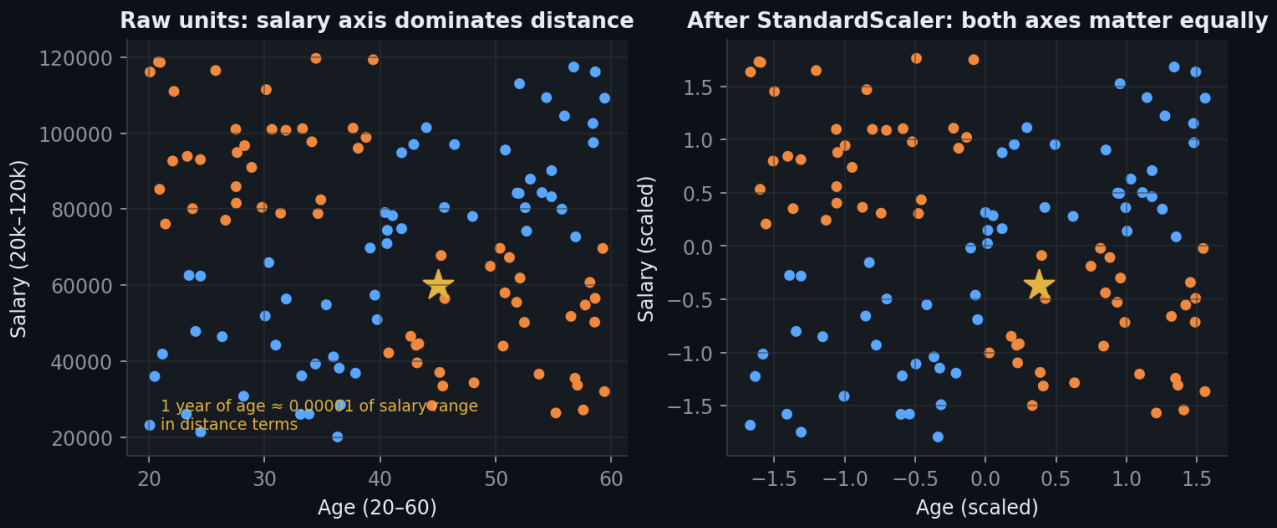


Figure 9 — Left: in raw units the salary axis is so large that the age axis is effectively flat. Right: after scaling, both features contribute equally to distances.

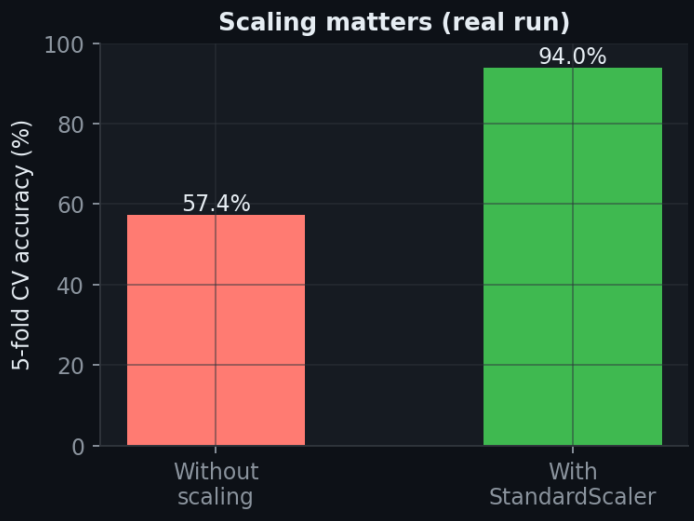


Figure 10 — Real experiment on a 6-feature dataset where two columns were inflated to the thousands: accuracy jumps from 57.4% to 94.0% just by scaling.

▲ Pro warning

Fit the scaler **only on training folds**. If you scale the whole dataset first and then cross-validate, information from the validation fold leaks into training (“data leakage”) and your scores become too optimistic. The fix is a `Pipeline` (shown in section 1.7), which refits the scaler inside every fold automatically.

1.7 Putting it together: the Grid Search setup

● **LEVEL: INTERMEDIATE**

Here is how you would structure these preferred values into a dictionary for a standard `GridSearchCV`. Combining these highly influential parameters into one dictionary allows the algorithm to systematically test every single combination to find the optimal architecture for your specific dataset:

param_grid (algorithm is left on 'auto')

```
param_grid = {
    'n_neighbors': [3, 5, 7, 9, 11, 15, 21],
    'weights': ['uniform', 'distance'],
    'p': [1, 2]
}
```

How big is this search? 7 values of K × 2 weights × 2 values of p = **28 combinations**. With 5-fold cross-validation that is 28 × 5 = **140 model fits** — tiny, because KNN "training" is instant.

The professional version (scaler inside a Pipeline)

knn_gridsearch.py

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import GridSearchCV, train_test_split
from sklearn.datasets import make_classification

X, y = make_classification(n_samples=600, n_features=10, n_informative=5,
                           n_redundant=2, flip_y=0.05, random_state=11)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

pipe = Pipeline([
    ('scaler', StandardScaler()),          # scaled inside every CV fold -> no leakage
    ('knn',    KNeighborsClassifier())     # algorithm='auto' by default
])

param_grid = {
    'knn__n_neighbors': [3, 5, 7, 9, 11, 15, 21],   # 'stepname__param' syntax
    'knn__weights':     ['uniform', 'distance'],
    'knn__p':           [1, 2],
}

gs = GridSearchCV(pipe, param_grid, cv=5, scoring='accuracy', n_jobs=-1)
gs.fit(X_train, y_train)

print(gs.best_params_, gs.best_score_)
print('Test accuracy:', gs.score(X_test, y_test))
```

Real output of the code above

```
best_params_ : {'knn__n_neighbors': 11, 'knn__p': 1, 'knn__weights': 'distance'}
best CV accuracy : 79.8 %
test accuracy    : 71.7 %
```

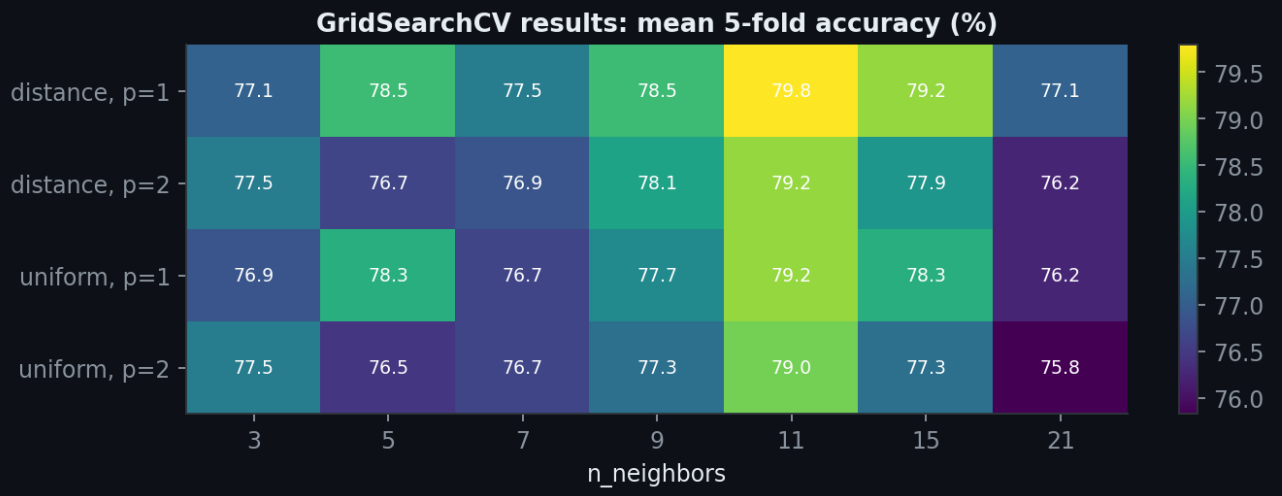


Figure 11 — All 28 combinations. Read it like a map: rows = (weights, p), columns = K. The best cell (K=11, distance, p=1) is not the same as the "textbook default", which is why we search.

> **● Reading the result honestly**
> The test score (71.7%) is lower than the CV score (79.8%) here. That is normal on a small, deliberately noisy synthetic dataset: the test set has only 120 rows, so its score has a large random error of several points. Pro habit: report the CV score **and** the test score, and never re-tune after peeking at the test set.

1.8 Pro level: KNN from scratch (≈15 lines)

● LEVEL: PRO

Writing it yourself proves you understand every hyperparameter above. Each line maps to one idea:

knn_from_scratch.py

```python
import numpy as np
from collections import Counter

def knn_predict(X_train, y_train, x_new, k=5, p=2, weights='uniform'):
    # step 2: distance to EVERY training point  ->  Minkowski with power p
    d = (np.abs(X_train - x_new) ** p).sum(axis=1) ** (1 / p)
    # step 3: indices of the k smallest distances  ->  n_neighbors
    idx = np.argsort(d)[:k]
    # step 4: voting  ->  weights
    votes = Counter()
    for i in idx:
        w = 1.0 if weights == 'uniform' else 1.0 / (d[i] + 1e-12)
        votes[y_train[i]] += w
    # step 5: winner
    return votes.most_common(1)[0][0]

# the worked example from section 1.1
Xtr = np.array([[1,1],[2,3],[4,4],[5,2],[6,5]]); ytr = np.array(['R','R','B','B','B'])
for k in (1, 3, 5):
    print(k, knn_predict(Xtr, ytr, np.array([3,3]), k=k))      # -> R, B, B
```

- **Cost:** one prediction touches all N training rows → O(N·d). This is the real reason KNN gets slow on huge data and why *algorithm* (kd_tree / ball_tree) exists.
- **Memory:** KNN stores the whole training set, so the "model file" is as big as your data.
- **Feature selection matters more than tuning:** irrelevant columns add noise to every distance. Removing useless features often beats any hyperparameter.

1.9 KNN cheat-sheet

Hyperparameter	Grid values	Small value / option →	Large value / option →	Why / tip
n_neighbors	[3,5,7,9,11,15,21] (+31)	Overfit, jagged, noise-sensitive	Underfit, smooth, blurry	Odd for binary; avoid k=1
weights	['uniform','distance']	uniform: equal votes	distance: closer = stronger	Helps imbalanced/messy data
p (metric=minkowski)	[1, 2]	1 = Manhattan (taxi)	2 = Euclidean (crow)	p=1 for many features / outliers
algorithm	— (keep 'auto')	speed only	speed only	Never changes accuracy
Scaling (not a parameter!)	StandardScaler / MinMaxScaler	—	—	**Mandatory**, put it in a Pipeline

PART 2 — Decision Trees

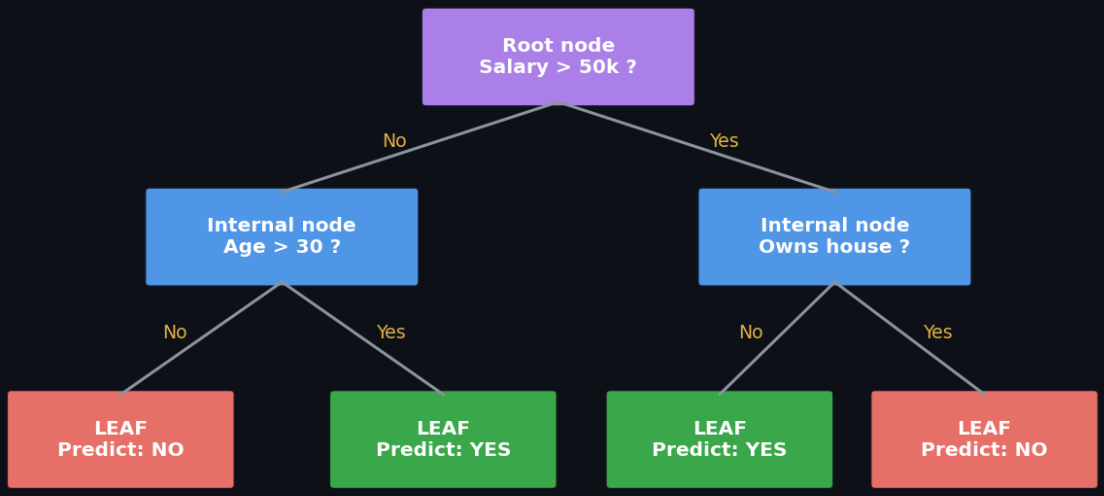
From “20 Questions” to cost-complexity pruning and an efficient grid search

2.1 The big idea: a flowchart that writes itself

● **LEVEL: BASIC**

> ☺ **Everyday analogy**
>
> Think of the game **20 Questions**. To guess an animal you ask “Does it fly?”, then “Is it bigger than a cat?” … Every answer narrows the possibilities. A decision tree is the same game, except the computer **learns the best questions automatically** from your data and writes them as a flowchart.

Anatomy of a decision tree (a flowchart of yes/no questions)



depth 0 = root, depth 1 = first split level, depth 2 = leaves here → max_depth=2 would stop exactly here.

Figure 12 — Anatomy. The root asks the first question; each internal node asks another; each leaf gives the final answer. The path from the root to a leaf is one ‘rule’, e.g. “Salary > 50k AND Owns house = Yes → YES”.

Word	Meaning
Root node	The first question, using the whole dataset.
Internal node	A later question (a split) on a subset of the data.
Leaf (terminal node)	No more questions. Predicts the majority class (classification) or the average value (regression) of the training samples that landed here.
Depth	How many questions you can chain from root to the deepest leaf. A tree of depth d has at most 2^d leaves.
Split / threshold	A question like Salary > 50,000. The tree only asks yes/no questions about **one feature at a time**.

2.2 How the tree chooses its questions (the math)

● **LEVEL: INTERMEDIATE**

The tree wants every answer to produce **purer** groups (groups where almost everybody belongs to one class). We measure “messiness” with an **impurity** score.

$$\text{Gini impurity: } G = 1 - \Sigma_k \, p_k^{\,2}$$

$$\text{Entropy: } H = -\Sigma_k \, p_k \cdot \log_2(p_k)$$

$$\text{Regression (MSE): } MSE = (1/n) \cdot \Sigma \, (y_i - \bar{y})^2$$

p_k = fraction of samples in the node that belong to class k. Both Gini and Entropy are 0 for a pure node and largest for a 50/50 mix.

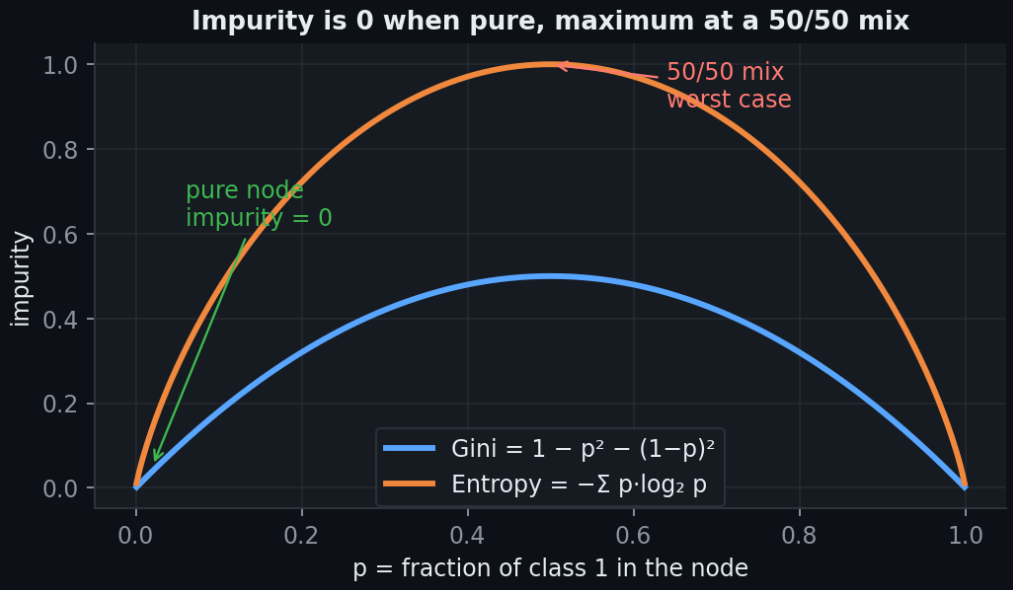


Figure 13 — Two classes. Impurity is 0 at the edges (pure) and highest in the middle (total confusion). Scikit-learn uses Gini by default.

$$\text{Impurity after a split: } G_{split} = (n_L/n) \cdot G_L + (n_R/n) \cdot G_R$$

$$\text{Information gain: } Gain = G_{parent} - G_{split} \rightarrow \text{choose the split with the largest gain}$$

The tree tries every feature and every threshold at each node, keeps the best, then repeats on each child (a greedy, step-by-step strategy).

Worked example

A node has 20 people: 10 buy (YES) and 10 don't (NO). The question “Salary > 50k?” sends 10 people left (2 YES, 8 NO) and 10 right (8 YES, 2 NO).

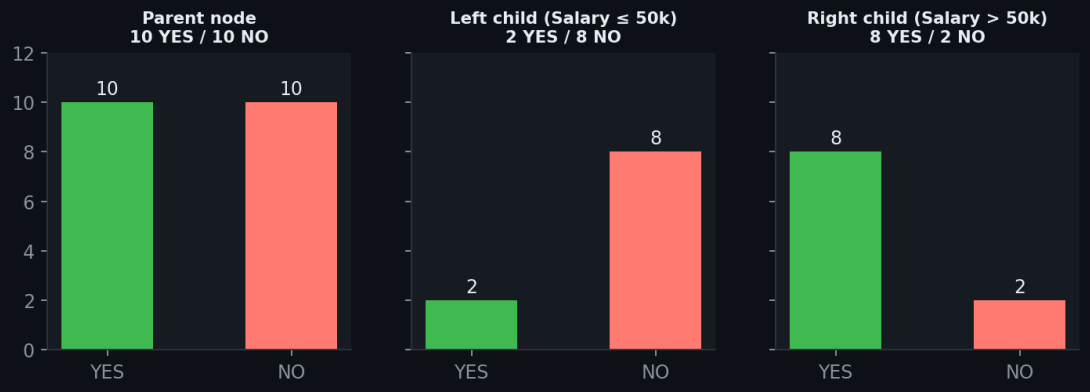


Figure 14 — Parent node (perfectly mixed) and the two children after the split.

$$G_{parent} = 1 - (0.5^2 + 0.5^2) = 0.50$$

$$G_{left} = 1 - (0.2^2 + 0.8^2) = 0.32 \quad G_{right} = 0.32$$

$$G_{split} = (10/20)\cdot 0.32 + (10/20)\cdot 0.32 = 0.32 \quad \text{Gain} = 0.50 - 0.32 = \mathbf{0.18}$$

(Entropy version: $H_{parent} = 1.00$, $H_{child} = 0.72$, Gain = 0.28)

The tree repeats this for hundreds of candidate questions and picks the one with the biggest gain. Then it recurses. **If nobody stops it, it keeps going until every leaf is perfectly pure** — and that is the root of all tuning below.

2.3 The core problem: trees memorise

● **LEVEL: BASIC**

Maximizing the efficiency and predictive accuracy of a single Decision Tree primarily revolves around **aggressively preventing its natural tendency to perfectly memorize the training data**. Because Decision Trees will grow indefinitely until every data point is isolated, tuning focuses on forcing the model to generalize.

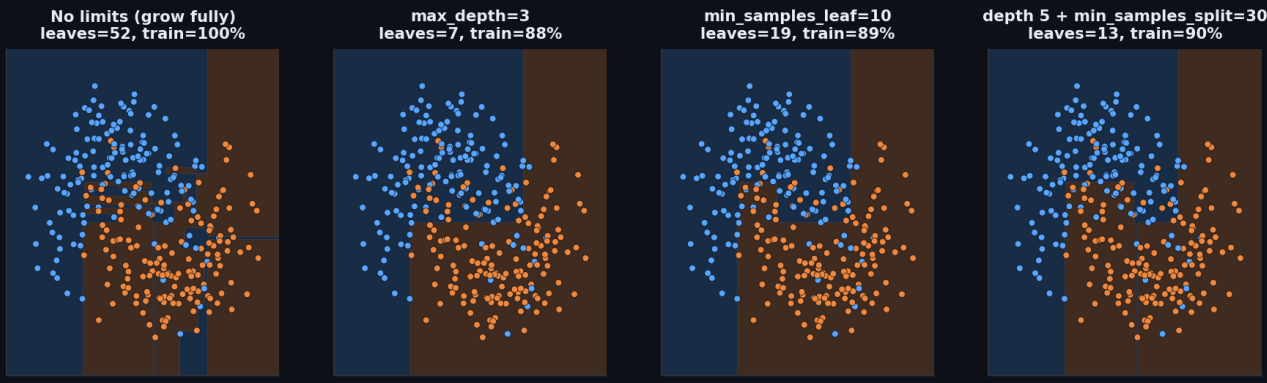


Figure 15 — Left: an unrestricted tree builds dozens of tiny rectangles to isolate every noisy point (100% train accuracy, awful generalisation). The other panels show pre-pruning limits producing simpler, smoother regions. Notice all borders are strictly horizontal or vertical.

2.4 Pre-Pruning (hyperparameter tuning)

● **LEVEL: INTERMEDIATE**

Pre-pruning means **stopping the tree early** with rules, before it becomes huge.

① max_depth — cap the number of questions

Limit max_depth: A tree left to its default settings will expand until all leaves are pure, guaranteeing severe overfitting. Cap the maximum depth (typically between **3 and 15**) to force the model to capture only the most dominant macro-patterns in your dataset.



Figure 16 — Real run on noisy data. Training accuracy climbs to 100% as the tree deepens, but test accuracy peaks very early (depth 2 here, 86.0%) and falls to about 79% at depth 20. Your job: find the peak.

② min_samples_split — how big must a node be to be allowed to split?

Increase min_samples_split: This dictates the minimum number of data points required in a node before the algorithm is allowed to branch it further. Raising this from the default of 2 to values like **10, 20, or 50** stops the tree from creating highly specific rules for microscopic clusters of data.

③ min_samples_leaf — how much evidence must each leaf have?

Set min_samples_leaf: This ensures that every final terminal node (leaf) has a minimum baseline of evidence (data points) backing its prediction. If set to 5, the tree will reject any split that would leave a leaf with 4 or fewer samples, effectively smoothing the decision boundaries and ignoring isolated outliers.

Parameter	Default	Typical grid	Controls	Too strict →
max_depth	None (unlimited)	[3, 5, 10, 15, None]	Height of the tree	Underfit (too simple)
min_samples_split	2	[2, 10, 20, 50]	Smallest node that may still split	Underfit
min_samples_leaf	1	[1, 5, 10, 20]	Smallest allowed leaf (evidence per prediction)	Underfit

★ Tip

These three overlap: a rule like min_samples_leaf=20 already prevents tiny splits, making a high min_samples_split partly redundant. That is fine for a grid search — but if time is limited, start with max_depth + min_samples_leaf.

2.5 Post-Pruning: Cost-Complexity Pruning (ccp_alpha)

● **LEVEL: PRO**

Tune ccp_alpha: Instead of stopping the tree from growing early, you allow it to grow into a massive, overfitted structure. Then, you apply Cost-Complexity Pruning (`ccp_alpha` in Scikit-Learn). This algorithm works backward, systematically snipping off the weakest branches that add complexity without significantly improving validation accuracy. Using Grid Search on `ccp_alpha` often yields the most mathematically robust single tree.

> ☺ **Everyday analogy**
>
> Pre-pruning is like telling a gardener “never grow taller than 1 metre”. Post-pruning lets the tree grow wild, then asks for each branch: “do your leaves justify your weight?” — and cuts the ones that don't.

$$R_\alpha(T) = R(T) + \alpha \cdot |T|$$

R(T) = training error (or impurity) of the tree |T| = number of leaves α ≥ 0 = price per leaf

The tree minimising R_α wins. α = 0 → no penalty → the full tree. Larger α → every leaf is expensive → smaller tree.

$$\text{Weakest-link score of a branch t: } \alpha_{eff}(t) = (R(t) - R(T_t)) / (|T_t| - 1)$$

= “error saved per extra leaf”. The branch with the smallest score is cut first; this repeats until only the root remains. Scikit-learn returns this whole ladder with cost_complexity_pruning_path().

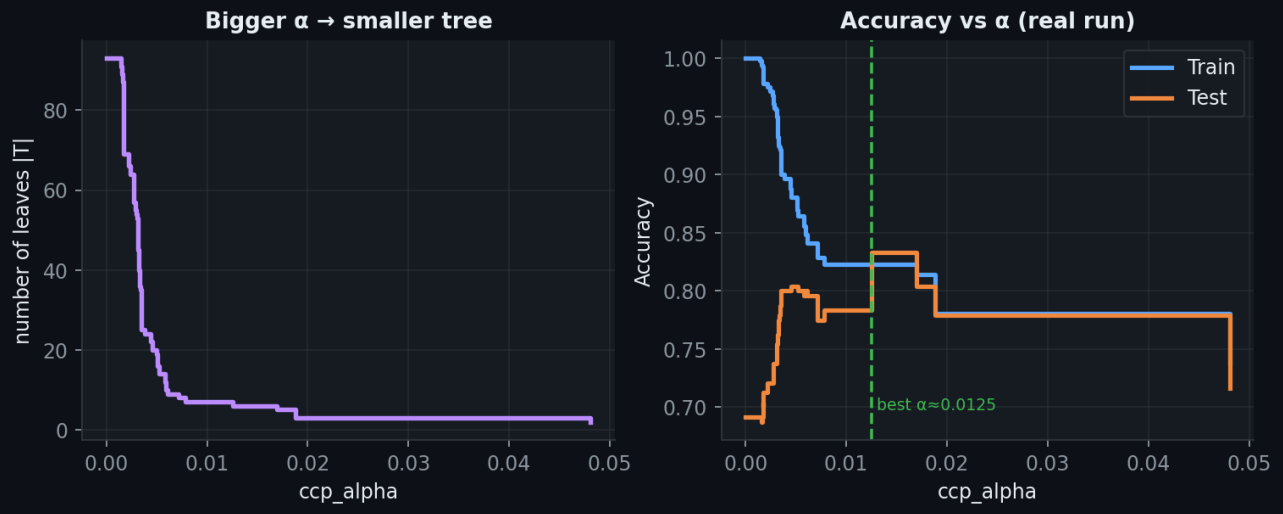


Figure 17 — Real run. Left: as α grows, leaves disappear. Right: the unpruned tree (α=0, 93 leaves) scores 69.2% on test data; pruning at α≈0.0125 leaves just 6 leaves and reaches 83.3%. Much simpler AND better.

How to build the ccp_alpha grid properly

Because the optimal pruning threshold is often microscopic, Stack Overflow users warn against using a random handful of numbers. Use numpy's `linspace` — e.g. `np.linspace(0, 0.05, 50)` — to systematically test tiny increments. This lets you find the exact point where the tree drops useless nodes without losing predictive power.

```
ccp_alpha_search.py
```

```python
import numpy as np
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import GridSearchCV

# Option A: the community recipe - a fine, evenly spaced ladder
param_grid = {'ccp_alpha': np.linspace(0, 0.05, 50)}

# Option B (pro): let the data tell you which alphas matter
path = DecisionTreeClassifier(random_state=42).cost_complexity_pruning_path(X_train, y_train)
param_grid = {'ccp_alpha': path.ccp_alphas[:-1]}      # last alpha = root only (useless tree)

gs = GridSearchCV(DecisionTreeClassifier(random_state=42), param_grid, cv=5, n_jobs=-1)
gs.fit(X_train, y_train)
print(gs.best_params_, gs.best_estimator_.get_n_leaves(), 'leaves')
```

● Clarification

The community phrase “prune nodes whose Gini is close to 0” is shorthand. Precisely, a branch is removed when the impurity it saves per extra leaf (the weakest-link score above) is smaller than α. Nodes that are already pure are the cheapest to remove, but the decision is about *saved error vs. added leaves*, not about a Gini cut-off.

▲ Pro warning

The useful α values depend on your dataset (in the run above the whole ladder ended at α≈0.048). If your grid only contains values larger than every α in the path, you will always get the 1-leaf tree. Check the path first.

2.6 Data preparation strategy

● LEVEL: INTERMEDIATE

Skip standardizing or scaling

Unlike KNN or Support Vector Machines, Decision Trees do not calculate physical distances. They search for binary thresholds (e.g., `Salary > $50,000`). Applying `StandardScaler` or `MinMaxScaler` wastes computational time and provides zero performance benefit to a tree-based model.

Why: scaling is an order-preserving transformation. “Salary > 50,000” and “scaled salary > 0.37” isolate exactly the same people, so the tree picks the same split.

Engineer diagonal relationships

Decision Trees can only draw strict horizontal and vertical boundaries on a graph. If the true pattern in your data is diagonal (e.g., a combination of two variables), the tree is forced to build a deep, computationally inefficient “staircase” of splits to approximate that line. You can radically improve efficiency by mathematically combining variables yourself (e.g., creating a `Debt_to_Income_Ratio` column) so the tree can capture the relationship with a single clean split.

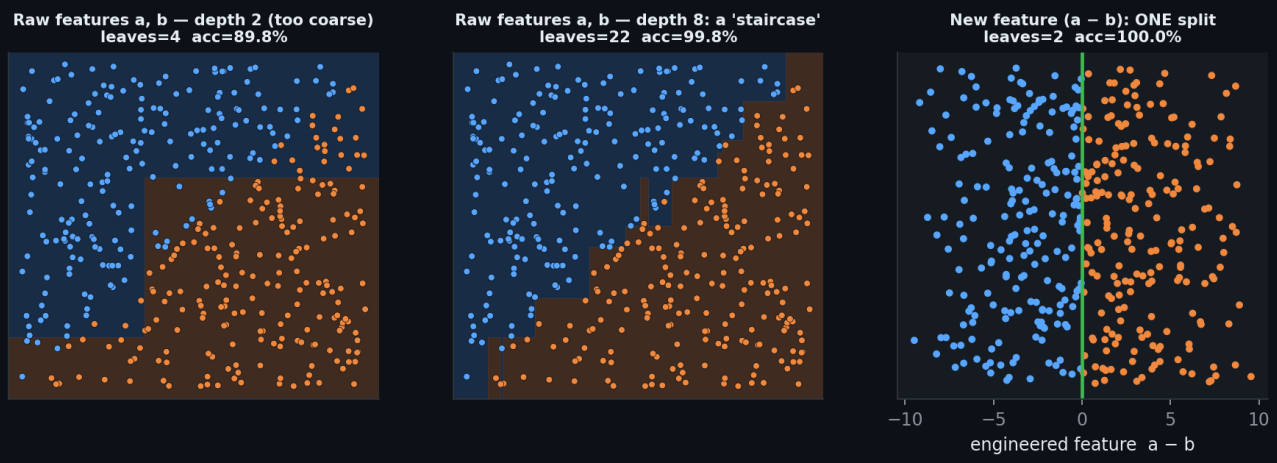


Figure 18 — The true rule is "a > b" (a diagonal line). Left/middle: on raw features the tree needs many leaves and still jags along the diagonal. Right: after adding one engineered feature (a − b) a single split is enough.

feature_engineering.py

```
df['Debt_to_Income_Ratio'] = df['Debt'] / df['Income']      # ratio
df['Price_per_sqft']       = df['Price'] / df['Area']       # ratio
df['Net_Worth']            = df['Assets'] - df['Liabilities']  # difference
```

2.7 Handling class imbalance

● **LEVEL: INTERMEDIATE**

Apply class_weight='balanced': If you are classifying data where one outcome represents 90% of the dataset and the other represents 10%, the tree will naturally ignore the 10% to achieve a quick 90% accuracy score. Balancing the weights forces the algorithm's math to treat a single misclassification in the minority class as severely as multiple misclassifications in the majority class, forcing it to hunt for patterns in the underrepresented data.

$$w_c = n_{samples} / (n_{classes} \times n_c)$$

Example, 1000 rows = 900 majority + 100 minority: $w_{majority} = 1000/(2 \cdot 900) = 0.56$, $w_{minority} = 1000/(2 \cdot 100) = 5.0$ → one minority mistake costs 9× more.

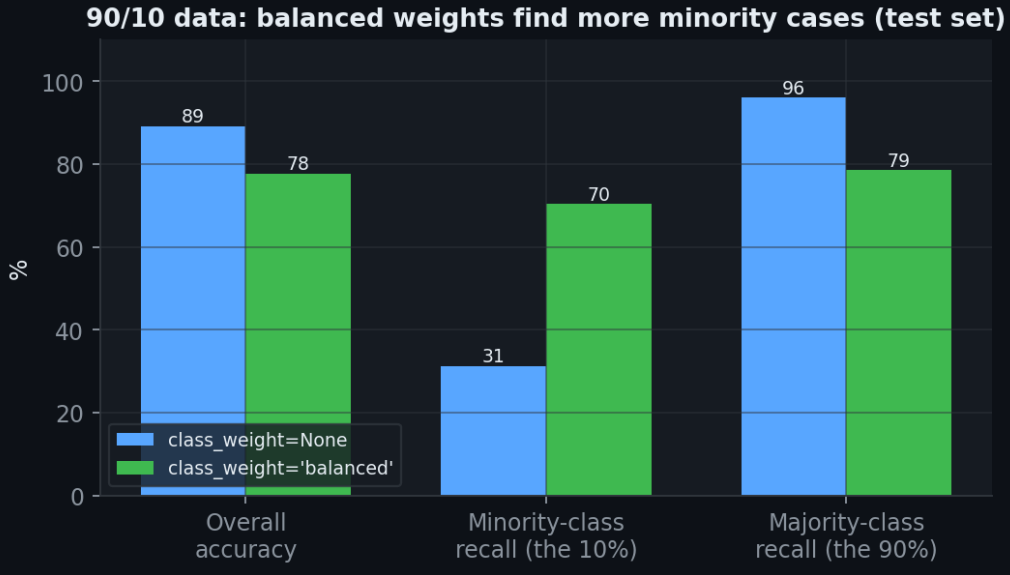


Figure 19 — Real run on a 90/10 dataset. Plain tree: 89% accuracy but it finds only 31% of the minority cases. Balanced tree: minority recall jumps to 70%, at the price of lower overall accuracy (78%).

> **▲ Pro warning**
> Do not judge an imbalanced problem by **accuracy** alone — the plain tree's 89% looks great while it misses 7 of every 10 rare cases. Use recall / precision / F1 (classification_report) or scoring='f1' / 'balanced_accuracy' in the grid search. Balancing is a *trade-off*: you accept more false alarms to catch more rare cases.

2.8 Efficiency tricks from the community (Stack Overflow & Reddit)

● LEVEL: PRO

Insights from Stack Overflow and Reddit machine learning communities reveal that making decision trees highly efficient relies on **reducing the mathematical search space during the splitting process**.

① Avoid the One-Hot Encoding trap

The community insight: On Stack Overflow, when users complain about a decision tree taking 15+ minutes to train on a relatively standard dataset, the culprit is almost always one-hot encoding.

How it works & affects the model: Decision trees must evaluate every possible split threshold for every single feature at every single node. If you take a categorical column (like "City") and one-hot encode it into 150 separate binary columns, you force the algorithm to calculate 150 times the math at every branch.

How to optimize: Group rare categories together into an "Other" bucket before encoding to keep the column count low. If you must use heavily encoded data, you must aggressively cap the `max_depth` early during model creation so the algorithm doesn't attempt to build deep branches out of sparse 0s and 1s.

Same data (5,000 rows, 1 categorical column 'City' with 150 values)	Columns	Training time
One-hot encoded, no limits	150	0.127 s
One-hot encoded + max_depth=5	150	0.015 s
Rare cities grouped into 'Other' (12 + Other), then one-hot	13	0.0019 s
Single ordinal-coded column	1	0.0048 s

Real timings on a small machine; the absolute values are tiny here, but the ratio (≈67× between 150 columns and 13 columns) is what scales up to the 15-minute complaints on large data.

② Randomize feature selection — max_features

The community insight: Both Reddit and Stack Overflow practitioners heavily advocate for changing the `max_features` parameter from its default setting to `'sqrt'` or `'log2'`.

How it works & affects the model: By default, a standard decision tree looks at 100% of your columns at every single branch to find the absolute mathematically perfect split. This exhaustive search is incredibly computationally expensive. Setting `max_features='sqrt'` forces the tree to only look at a random subset (the square root of the total columns) at each node.

How to optimize: Add `['sqrt', 'log2', None]` to your Grid Search. Not only does this drastically cut down training time, but it also acts as a powerful anti-overfitting tool by preventing

the tree from over-relying on one aggressively dominant feature.

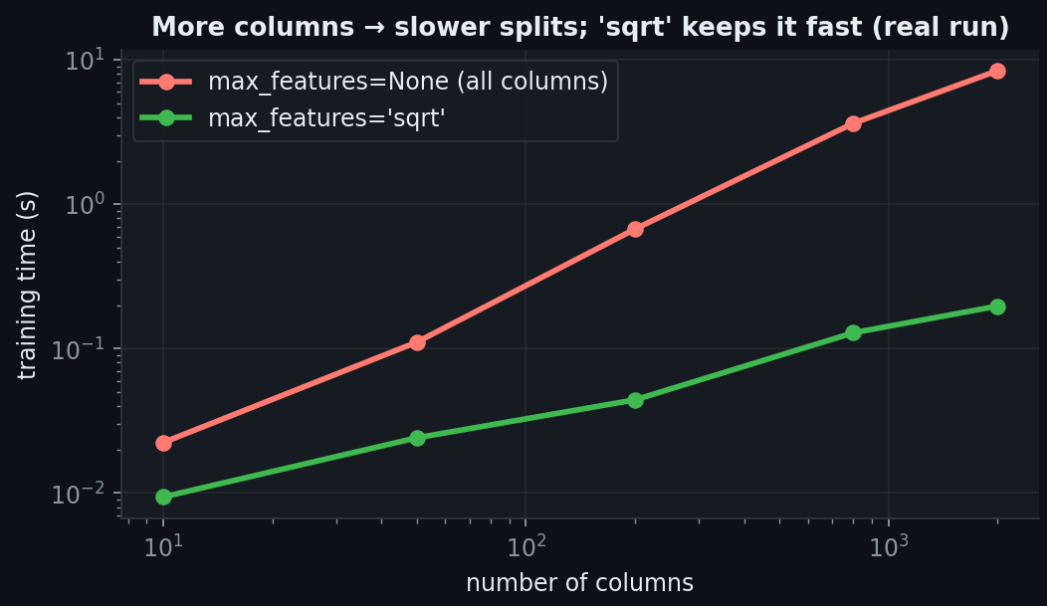


Figure 20 — Real run: with 2,000 columns the full search takes about 8.4 s versus about 0.2 s for 'sqrt' (≈43× faster). With 100 columns, sqrt looks at only 10 per node.

sqrt: m = ⌊√p⌋ (p = 100 columns → 10) log2: m = $\lfloor \log_2 p \rfloor$ (p = 100 → 6)

m = number of randomly chosen columns examined at each split. This is the same idea that powers Random Forests.

③ Precision pruning with ccp_alpha

The community insight: The most discussed parameter for structural efficiency on Stack Overflow is `ccp_alpha` (Cost-Complexity Pruning). Reddit discussions also emphasize that pruning based on the Gini index is vastly superior to hard-stopping a tree.

How it works & affects the model: Instead of blindly guessing the right `max_depth` upfront, you let the tree grow large, then prune branches that do not significantly improve the overall accuracy.

How to optimize: When building a Grid Search for `ccp_alpha`, use numpy's `linspace` — e.g. `np.linspace(0, 0.05, 50)` — to test tiny increments. (Full explanation, math and a real run are in section 2.5.)

④ Enforce min_impurity_decrease for sparse trees

The community insight: Reddit discussions on "Globally Optimum Sparse Decision Trees" highlight the massive inefficiency of "complete" trees — where the algorithm forces splits simply because it hasn't hit a depth limit yet.

How it works & affects the model: This parameter acts as a strict ROI (Return on Investment) threshold for the algorithm. It tells the tree, "Do not calculate and create a new branch unless it improves the purity of the node by at least X."

How to optimize: Set this to a small fraction (e.g., 0.01 or 0.05) in your Grid Search. It forces the tree to remain "sparse" (structurally smaller and faster) by stopping the algorithm from wasting processing power on microscopic, irrelevant branches that only separate a few outlier data points.

split is allowed only if: $(N_t/N) \cdot (G_t - (N_L/N_t)\cdot G_L - (N_R/N_t)\cdot G_R) \geq$ min_impurity_decrease

N = all training samples, N_t = samples in this node. The gain is weighted by the node's share of the data, so it is an absolute number — not literally a percentage.

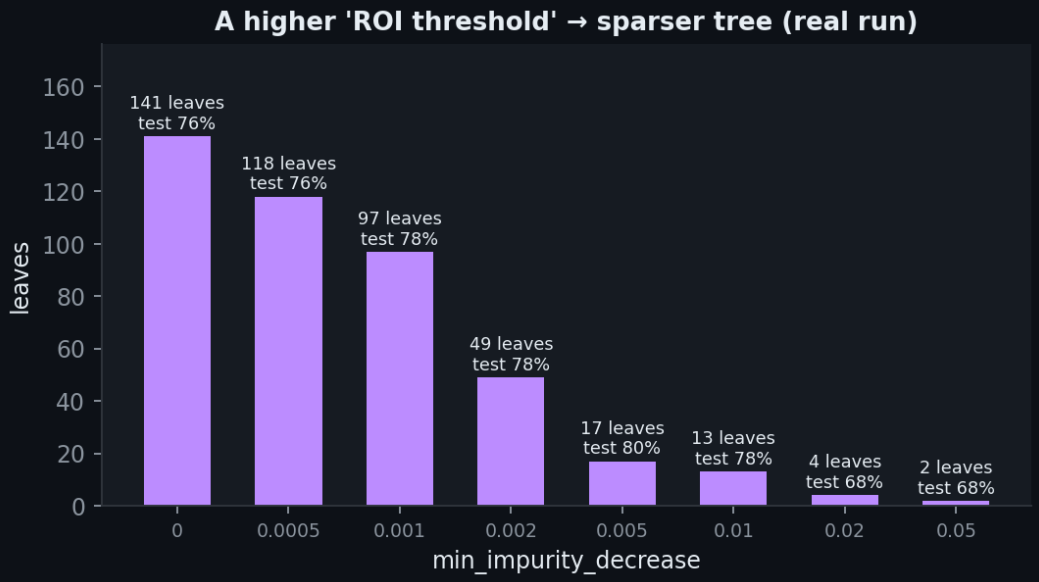


Figure 21 — Real run on 1,050 training rows. 0 (default): 141 leaves. 0.005: 17 leaves and the best test score (80%). 0.02 and 0.05: only 4 and 2 leaves and accuracy collapses to 67.6%.

▲ Pro warning

The thresholds 0.01 and 0.05 from the community advice are **dataset-dependent**. Because Gini at the root is at most 0.5, a value of 0.05 can be so strict that the tree stops after one or two splits (underfitting, see the right side of Figure 21). Always include 0.0 and a few small values (0.0005–0.01) in the grid.

2.9 The complete tuned pipeline — your code, explained line by line

● **LEVEL: PRO**

dt_gridsearch.py (your script, unchanged)

```
import pandas as pd
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import classification_report, accuracy_score
from sklearn.datasets import make_classification

# 1. Prepare the Data (Using a synthetic dataset for demonstration)
X, y = make_classification(n_samples=1000, n_features=20, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 2. Initialize the Base Model
# Setting a random_state ensures reproducibility across different runs
dt_classifier = DecisionTreeClassifier(random_state=42)

# 3. Define the Hyperparameter Grid
# This incorporates the efficiency parameters (max_features, min_impurity_decrease)
param_grid = {
    'max_depth': [3, 5, 10, 15, None],
    'min_samples_split': [2, 10, 20, 50],
    'min_samples_leaf': [1, 5, 10, 20],
    'max_features': [None, 'sqrt', 'log2'],
    'min_impurity_decrease': [0.0, 0.01, 0.05]
}

# 4. Configure GridSearchCV
# cv=5 executes 5-fold cross-validation for robust scoring
# n_jobs=-1 utilizes all available CPU cores to drastically reduce training time
grid_search = GridSearchCV(
    estimator=dt_classifier,
    param_grid=param_grid,
    cv=5,
    scoring='accuracy',
    n_jobs=-1,
    verbose=1
)

# 5. Execute the Grid Search
grid_search.fit(X_train, y_train)

# 6. Extract and Evaluate the Best Model
best_tree_model = grid_search.best_estimator_

print("Best Hyperparameters Found:")
for param, value in grid_search.best_params_.items():
    print(f" - {param}: {value}")
print(f"\nBest Cross-Validation Accuracy: {grid_search.best_score_ * 100:.2f}%")

# 7. Test the Optimized Model on Unseen Data
y_pred = best_tree_model.predict(X_test)
print(f"\nTest Set Accuracy: {accuracy_score(y_test, y_pred) * 100:.2f}%")
print("\nClassification Report:\n", classification_report(y_test, y_pred))
```

Step	What it does	Why it matters
1 Data	make_classification builds 1000 fake rows with 20 features; train_test_split keeps 20% (200 rows) locked away.	The test set must stay unseen until the very end.
2 Base model	A DecisionTreeClassifier with random_state=42.	Random tie-breaking and max_features sampling become repeatable.

Step	What it does	Why it matters
3 Grid	5 × 4 × 4 × 3 × 3 = **720 combinations**.	The grid mixes pre-pruning (depth, split, leaf), speed/regularisation (max_features) and sparsity (min_impurity_decrease).
4 GridSearch CV	5-fold CV for every combination = 720 × 5 = **3,600 fits**.	Each score is an average over 5 different validation folds, not one lucky split.
5 fit	Runs all 3,600 fits in parallel.	After the search it refits the winner on the whole training set.
6 Best model	best_estimator_ is already trained.	You can call .predict() immediately.
7 Test	Accuracy + precision/recall/F1 per class on the 200 held-out rows.	The honest, final grade.

Key implementation details

Setting	What it does
n_jobs=-1	**Critical when running a large parameter grid.** It parallelizes the workload across your machine's processor cores. Without this, Grid Search will run sequentially on a single core, taking significantly longer.
verbose=1	Prints real-time updates to your console while the Grid Search is running (e.g., “Fitting 5 folds for each of N candidates, totalling M fits”). This confirms the model hasn't frozen during long processing times.
random_state	Declaring this in the base estimator ensures that the randomized feature selection (max_features) and node splits are identical if you run the exact same script twice, allowing you to accurately track improvements.
grid_search.best_estimator_	The GridSearchCV object automatically refits the model on the entire training dataset using the single best combination of hyperparameters it found. You do not need to manually retrain the model; you can immediately call .predict() on the returned estimator.

● Small correction

The script's own comment shows “240 candidates, 1200 fits” as an *example* of the verbose message. For the grid in this script the real numbers are **720 candidates and 3,600 fits** (verified when I ran it).

What I got when I ran it

```
Real output (scikit-learn 1.8, random_state=42)

Fitting 5 folds for each of 720 candidates, totalling 3600 fits
Best Hyperparameters Found:
 - max_depth: 5
 - max_features: None
 - min_impurity_decrease: 0.01
 - min_samples_leaf: 1
 - min_samples_split: 2

Best Cross-Validation Accuracy: 89.88%
Test Set Accuracy: 87.50%

              precision    recall  f1-score   support
           0       0.82      0.94      0.87        93
           1       0.94      0.82      0.88       107
    accuracy                           0.88       200
   macro avg       0.88      0.88      0.87       200
weighted avg       0.88      0.88      0.88       200
```

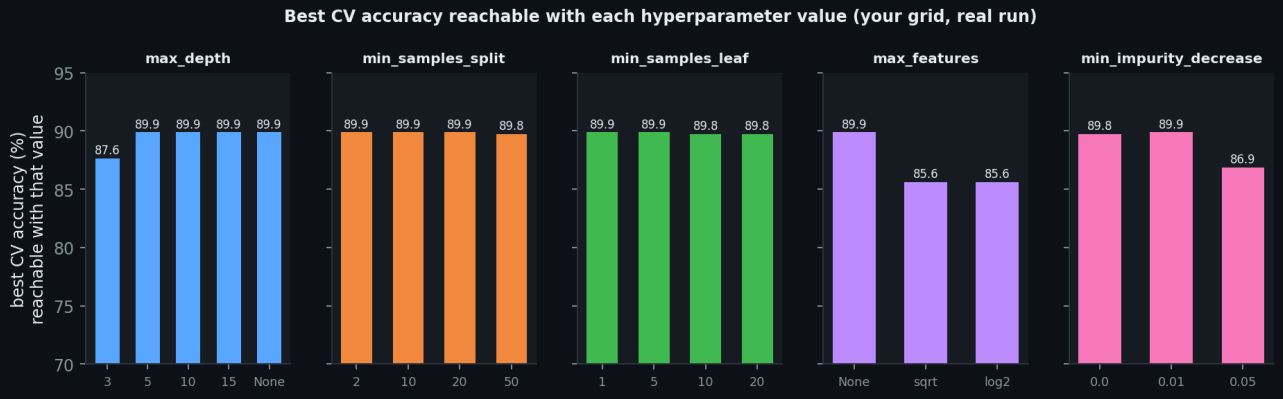


Figure 22 — For each parameter value: the best CV accuracy that any combination containing it reaches. max_depth=3 and min_impurity_decrease=0.05 are clearly too strict (they cap accuracy lower); the other values can all reach ≈90% — the grid has several near-equivalent winners.

◆ Real run (executed while building this tutorial)

An un-tuned, fully grown tree scored **the same 87.5%** on this test set — but it had **56 leaves**, while the tuned tree has only **5 leaves** (depth 5). The honest lesson: on this easy dataset tuning did not raise accuracy, it made the model **~11× smaller, faster, and far easier to explain** with the same score. Note also that max_features=None won, because with only 20 columns the speed trick has little to gain — it matters most when you have hundreds or thousands of columns (Figure 20).

2.10 Pro toolbox: look inside your tree

● **LEVEL: PRO**

inspect_tree.py

```
from sklearn.tree import export_text, plot_tree
import matplotlib.pyplot as plt

print(export_text(best_tree_model, feature_names=[f'f{i}' for i in range(20)], max_depth=3))

plt.figure(figsize=(14, 6))
plot_tree(best_tree_model, filled=True, max_depth=3, fontsize=7)
plt.show()

# which features did the tree actually use?
import numpy as np
imp = best_tree_model.feature_importances_
print(np.argsort(imp)[::-1][:5], np.sort(imp)[::-1][:5])
```

- **Recommended tuning order:** (1) fix the data (encode wisely, engineer ratios, no scaling needed) → (2) coarse grid on max_depth / min_samples_leaf → (3) add max_features + min_impurity_decrease → (4) fine ccp_alpha search around the best tree → (5) judge on the right metric (F1/recall if imbalanced).
- **Keep the grid small:** the number of fits multiplies — 5 parameters of 3–5 values each already means thousands of fits. Use RandomizedSearchCV or tune in stages when the grid explodes.
- **Single trees are unstable:** a tiny change in data can change the top split and the whole tree. When you need more accuracy, the natural next step is an ensemble (Random Forest, Gradient Boosting) — which reuses every idea in this chapter.

2.11 Decision Tree cheat-sheet

Goal	Hyperparameter / action	Typical values
Stop memorising	max_depth	3 – 15 (or None to be pruned later)
Block tiny splits	min_samples_split	10, 20, 50 (default 2)
Evidence per leaf	min_samples_leaf	5, 10, 20 (default 1)
Train faster / fight dominance	max_features	'sqrt', 'log2', None
Sparse, small tree	min_impurity_decrease	0.0, small fractions (0.0005 – 0.01); 0.05 can be too strict
Best-shaped single tree	ccp_alpha	np.linspace(0, 0.05, 50) or the pruning path
Imbalanced classes	class_weight	'balanced' (judge by recall / F1)
Data prep	No scaling · engineer ratios · group rare categories before one-hot	—

PART 3 — Wrap-up

Compare, avoid mistakes, practise

3.1 KNN vs Decision Tree at a glance

	KNN	Decision Tree
How it predicts	Votes of the K closest stored points	Follows yes/no questions to a leaf
Training	None (“lazy”) — just stores data	Builds the tree (greedy splits)
Prediction speed	Slow on big data (compares to many points)	Very fast (a few comparisons)
Needs scaling?	**Yes — mandatory**	**No** — useless work
Main overfitting dial	n_neighbors (small = overfit)	max_depth, min_samples_*, ccp_alpha
Decision boundary	Smooth, any shape	Axis-parallel rectangles (staircase on diagonals)
Interpretability	Low (“because of the neighbours”)	High (readable rules)
Weak spot	Many features / curse of dimensionality	Unstable & prone to overfit when unpruned
Typical grid size	28 combinations	hundreds of combinations

3.2 Ten mistakes to avoid

- Using KNN without scaling the features (Figure 10: 57% → 94%).
- Fitting the scaler on the full data before cross-validation (leakage) — use a Pipeline.
- Including k=1 in the KNN grid, or an even K for binary classification.
- Spending grid-search time on `algorithm` — it never changes KNN accuracy.
- Scaling features for a Decision Tree (zero benefit).
- Judging an imbalanced problem by accuracy alone.
- Leaving a tree unpruned (default) and trusting its 100% training score.
- Choosing a ccp_alpha grid that does not match the dataset's pruning path.
- Tuning on the test set (peeking) — the test set is used exactly once.
- One-hot encoding a 150-category column and then wondering why the tree is slow.

3.3 Practice exercises

● **LEVEL: PRO**

- **Exercise 1 (by hand):** In the section 1.1 example, move Q to (5, 3). Recompute distances. What do k=1, 3, 5 predict?

- **Exercise 2:** Take the KNN pipeline in section 1.7 and delete the scaler. How far does the CV score fall?
- **Exercise 3:** A node has 6 YES and 2 NO and splits into (5 YES, 0 NO) and (1 YES, 2 NO). Compute the Gini gain.
- **Exercise 4:** Run the ccp_alpha search (section 2.5) on your own data. Compare leaves and test score of the unpruned tree and the best pruned tree.
- **Exercise 5:** On a dataset with 300+ columns, time max_features=None vs 'sqrt' with and without n_jobs=-1.
- **Exercise 6 (pro):** Re-run the Decision Tree grid with scoring='f1' on a 90/10 dataset and add class_weight=[None,'balanced'] to the grid.

Answers hints: Ex.3 → $G_{parent} = 1-(0.75^2+0.25^2) = 0.375$; $G_{left} = 0$; $G_{right} = 1-(1/3^2 + 2/3^2) = 0.444$; $G_{split} = (5/8)\cdot 0 + (3/8)\cdot 0.444 = 0.167$; Gain = 0.208.

Appendix — coverage checklist of your original script

Item in your script	Where it is in this tutorial
KNN: n_neighbors (what, tips, values [3…21], avoid k=1, odd numbers) + expanded version (how it works / helps / affects / values incl. 31)	§1.2
KNN: weights (uniform / distance, definitions, helps, affects, imbalanced data)	§1.3
KNN: metric and p (Manhattan, Euclidean, Minkowski default, curse of dimensionality, outliers)	§1.4
KNN: algorithm (ball_tree, kd_tree, brute, auto — exclude from grid)	§1.5
KNN: standard grid-search dictionary + scaling warning (StandardScaler / MinMaxScaler, salary vs age)	§1.6, §1.7
DT: pre-pruning — max_depth, min_samples_split, min_samples_leaf	§2.4
DT: post-pruning — ccp_alpha	§2.5
DT: data preparation — skip scaling, engineer diagonal relationships (Debt_to_Income_Ratio)	§2.6
DT: class imbalance — class_weight='balanced'	§2.7
Community: one-hot trap, max_features, precision pruning with ccp_alpha + linspace, min_impurity_decrease	§2.8
Full GridSearchCV code (7 steps) + key implementation details (n_jobs, verbose, random_state, best_estimator_)	§2.9